Unordered Map | Ordered Map

Unordered Map

Definition

An unordered map is a collection of **key–value pairs** with **random order**. **Each key is unique**, and values are accessed using their corresponding keys. It is implemented using a hash table.

Key Properties

a) Keys are unique.
b) Not sorted order depends on hash values.
c) Faster operations **O(1)** for **insertion**, **deletion**, and **lookup**.

Ordered Map

Definition

An ordered map is a collection of **key–value pairs** that are **sorted based on the keys**. **Each key is unique**, and the elements are stored in sorted order. It is implemented using a BST.

Key Properties

a) Keys are unique.
b) Sorted order of keys (usually ascending).
c) Slower operations **O(log n)** for **insertion**, **deletion**, and **lookup**.

Unordered Map | Ordered Map | Common Operations

1) Declaration + Allocation

Unordered Map (hash-based, no sorting)

Signature
`Unordered_map<key_type,value_type> map_name`

```
1) unordered_map<int,string> myMap1;

2) unordered_map<int,string> myMap2 = {{1, "One"},{2, "Two"}}
```

Ordered Map (BST-based, sorted))

Signature
`ordered_map<key_type,value_type> map_name`

```
1) ordered_map<int,string> myMap1;

2) ordered_map<int,string> myMap2 = {{1, "One"},{2, "Two"}}
```

Unordered Map| Ordered Map| Common Operations

2) Adding / Updating an element

Same Signature for Both

```
1) map1[Key] = value;
```

3) Adding all elements

Same Signature for Both

```
1) map1.insert(map2.begin(), map2.end());
```

4) Remove an element

Same Signature for Both

```
1) map1.erase(key);
```

5) Getting the Size

Same Signature for Both

```
1) map1.size();
```

Unordered Map | Ordered Map | Common Operations

6) Two Maps are equal or not

Same Signature for Both

```
1) map1 == map2; → Boolean
```

7) Checking Existence of Key

Same Signature for Both

```
1) map1.find(key) != map1.end();
2) map1.count(key);
```

→ iterator
→ 1/0 size_t/int

8) Looping in Map

Same for Both

Accessing Only Keys

```
1) map<int, int> myMap = {{1, 10},{2, 20},{3, 30},{4, 40}};
2) for(auto pair : myMap) {
3)   cout << pair.first << endl;}
```

→ {1,2,3,4}

Accessing Only Values

```
1) map<int, int> myMap = {{1, 10},{2, 20},{3, 30},{4, 40}};
2) for(auto pair : myMap) {
3)   cout << pair.second << endl;}
```

→ {10,20,30,40}

8) Looping in Map

Same for Both

Accessing both (Keys and Values)

```
1) map<int, int> myMap = {{1, 10},{2, 20},{3, 30},{4, 40}};
2) for(auto pair : myMap) {
3)     cout << pair.first << " " << pair.second << endl;
```

8) Looping in Map

Same for Both

Accessing both (Keys and Values)

```
1) map<int, int> myMap = {{1, 10},{2, 20},{3, 30},{4, 40}};
2) for(auto pair : myMap) {
3)     cout << pair.first << " " << pair.second << endl;
```

Unordered Map| Ordered Map| Common Operations

9) Map Creation

Unordered map

```
1) vector<int> vec = {1,2,2,3,5,6,1,8};
2) unordered_map <int, int> myMap;
3) for (auto num : vec)
4)    {myMap[num] += 1;}
```

{{8,1},{6,1},{2,2},{1,2},{5,1},{3,1}}

Ordered map

```
1) vector<int> vec = {1,2,2,3,5,6,1,8};
2) map <int, int> myMap;
3) for (auto num : vec)
4)    {myMap[num] += 1;}
```

{{1,2},{2,2},{3,1},{5,1},{6,1},{8,1}}

Unordered Map| Ordered Map| Common Operations

9) Map Creation

Ordered map

Ascending order

```
1) vector<int> vec = {1,2,2,3,5,6,1,8};
2) map <int, int> myMap; {{1,2},{2,2},{3,1},{5,1},{6,1},{8,1}}
3) for (auto num : vec)
4)   {myMap[num] += 1;}
```

Descending order

```
1) vector<int> vec = {1,2,2,3,5,6,1,8};
2) map <int, int, greater<int>> myMap;
3) for (auto num : vec)  {{8,1},{6,1},{5,1},{3,1},{2,2},{1,2}}
4)   {myMap[num] += 1;}
```

10) Sorting of a Map

1) Sorting by Keys

- a) Ascending Ordered
- b) Descending Ordered

2) Sorting by Values

- a) Ascending Ordered
- b) Descending Ordered

3) Custom Sorting

- a) Ascending Ordered
- b) Descending Ordered



Unordered Map| Ordered Map| Common Operations

10) Sorting of a Map | 1) Sorting by Keys | Ascending order

Custom function

```
1) bool fun(pair<int, string> P1, pair<int, string> P2)

2)  {return P1.first < P2.first;}
```

Unordered map

```
3) unordered_map<int,string>Map = {{10,"AB"},{30,"CD"},{8,"DE"}};
```

Converts into a pair of vector

```
4) vector< pair< int, string > > vec(Map.begin(),Map.end());
```

Sort by keys

```
5) sort(vec.begin(), vec.end(), fun);
```

```
Map = {{8,"DE"},{10,"AB"},{30,“CD"}
```

Unordered Map| Ordered Map| Common Operations

10) Sorting of a Map | 1) Sorting by Keys | Descending order

Custom function

```
1) bool fun(pair<int, string> P1, pair<int, string> P2)

2)  {return P1.first > P2.first;}
```

Unordered map

```
3) unordered_map<int,string>Map = {{10,"AB"},{30,"CD"},{8,"DE"}};
```

Converts into a pair of vector

```
4) vector< pair< int, string > > vec(Map.begin(),Map.end());
```

Sort by keys

```
5) sort(vec.begin(), vec.end(), fun);
```

```
Map = {{30,"CD"},{10,"AB"},{8,"DE"}
```

Unordered Map| Ordered Map| Common Operations

10) Sorting of a Map | 2) Sorting by Values | Ascending order

Custom function

```
1) bool fun(pair<int, string> P1, pair<int, string> P2)

2)   {return P1.second < P2.second;}
```

Unordered map

```
3) unordered_map<int,string>Map = {{10,"AB"},{30,"CD"},{8,"DE"}};
```

Converts into a pair of vector

```
4) vector< pair< int, string > > vec(Map.begin(),Map.end());
```

Sort by keys

```
5) sort(vec.begin(), vec.end(), fun);
```

```
Map = {{10,"AB"},{30,"CD"},{8,"DE"}
```

Unordered Map| Ordered Map| Common Operations

10) Sorting of a Map | 2) Sorting by Values | Descending order

Custom function

```
1) bool fun(pair<int, string> P1, pair<int, string> P2)

2)   {return P1.second > P2.second;}
```

Unordered map

```
3) unordered_map<int,string>Map = {{10,"AD"},{30,“AB"},{8,"DE"}};
```

Converts into a pair of vector

```
4) vector< pair< int, string > > vec(Map.begin(),Map.end());
```

Sort by keys

```
5) sort(vec.begin(), vec.end(), fun);
```

```
Map = {{8,"DE"},{30,"CD"},{10,"AB"}
```

Unordered Map| Ordered Map| Common Operations

10) Sorting of a Map | 3) Custom Sorting | Descending order

Custom function

```
1) bool fun(pair<int, string> P1, pair<int, string> P2)
2) if (P1.second != P2.second)
3)   {return P1.second < P2.second;}
4) {return P1.first > P2.first;}
```

Sort by values,
if the values are the same,
then sort by keys.

Unordered map

```
3) unordered_map<int,string>Map = {{10,"AB"},{30,"AB"},{8,"DE"}};
```

Converts into a pair of vector

```
4) vector< pair< int, string > > vec(Map.begin(),myMap.end());
```

Sort by keys

```
5) sort(vec.begin(), vec.end(), fun);
```

```
Map = {{30,"AB"},{10,"AB"},{8,"DE"}
```

1	**Squared Values**
Problem Statement	Given N integers. Create a dictionary with the integers as keys and the square of the integers as values. Print the key (in sorted order) and the associated value separated by space.
I/P Format	First line contains the value of N. The next line contains N space separated integers.
O/P Format	N lines of output. Each line contains two space separated integers.
Constraint	1 <= N <= 10^5 -10^6 <= arr[i] <= 10^6
Sample Input	5 12 13 1 2 5
Sample Output	1 1 2 4 5 25 12 144 13 169

2	**Unique Non Unique**
Problem Statement	Given an array with N integers. Print the product of count of unique and non-unique elements. The expected complexity is O(N).
I/P Format	First line contains the value of N. The second line contains N space separated integers.
O/P Format	Single integer in a line.
Constraint	1 <= N <= 10^5 -10^6 <= arr[i] <= 10^6
Sample Input	11 10 2 3 4 2 5 6 4 2 3 6
Sample Output	8
Explanation	Unique integers are 10, and 5, and non unique integers are, 2, 3, 4, and 6. The product of their counts is 8. Hence the answer.

3	**Frequency Sorted**
Problem Statement	Given a list of integers, having N elements. Print the integers in decreasing order of their frequency of occurrence in the list. Each integer should appear only once in the output. In case of a tie, print the smaller element first. See sample for better understanding.
I/P Format	First line contains the value of N. The second line contains N space separated integers.
O/P Format	Space separated integers.
Constraint	`1 <= N <= 10^6` `-10^6 <= arr[i] <= 10^6`
Sample Input	7 `3 9 3 2 5 2 3`
Sample Output	`3 2 5 9`

4	**Naming Ceremony**
Problem Statement	Witto wants to name his newborn daughter and son. He has got hold of a list of names according to his belief system. He wants to name his daughter by the name that appears least in his list of names and he wants to name his son by the name that appears most in the list. In case of a tie he will pick the lexicographically smaller name. Help Witto find suitable names for his daughter and son.
Constraint	`1 <= N <= 10^5` `1 <= names[i]<= 10^3`
Sample Input	`7` `Jackal Tiger Jackal Fox Lion Eagle Lion`
Sample Output	`Eagle Jackal`
Explanation	Tiger, Eagle, and Fox appear only once. Eagle is lexicographically the smallest. Jackal and Lion comes twice. Jackal is lexicographically the smallest Hence the answer is Eagle followed by Jackal.